

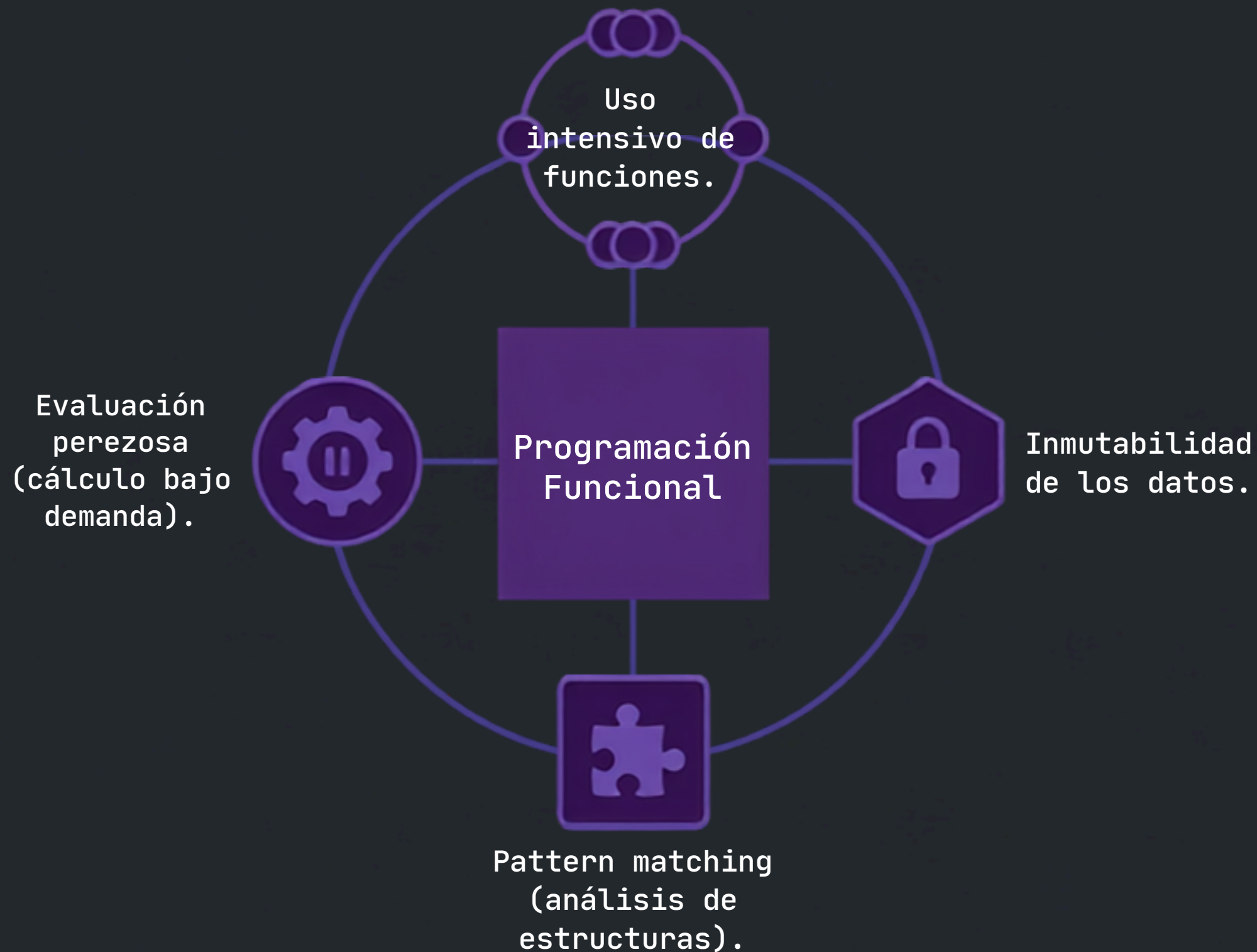
Comparación de Haskell y F#

Implementación y rendimiento de Bubble Sort y Quick Sort

Universidad Tecnológica Nacional - UTN BA
Sintaxis y semántica de los lenguajes - K2002
Ing. Roxana Leituz

Nicole Brunstein, Maximo Colombatto,
Ariana Castro, Federico Dimentsein,
Agustina Marques Serra
Fecha: 04/08/2026

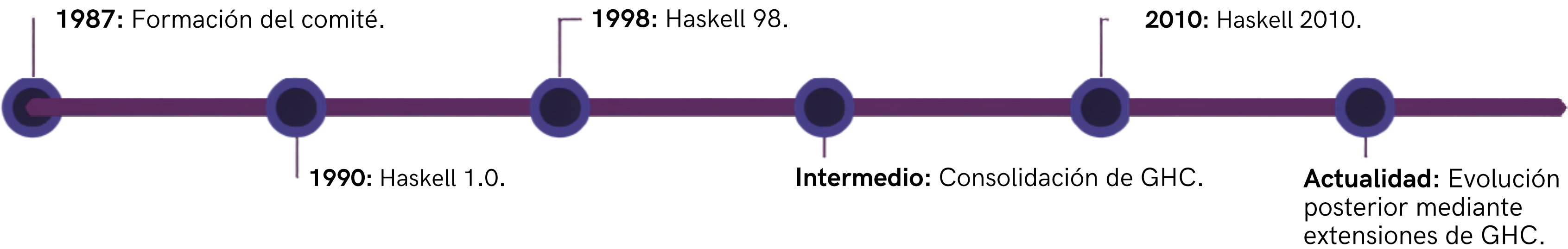
Origen de Haskell



El Problema y la Solución (Década de 1980)

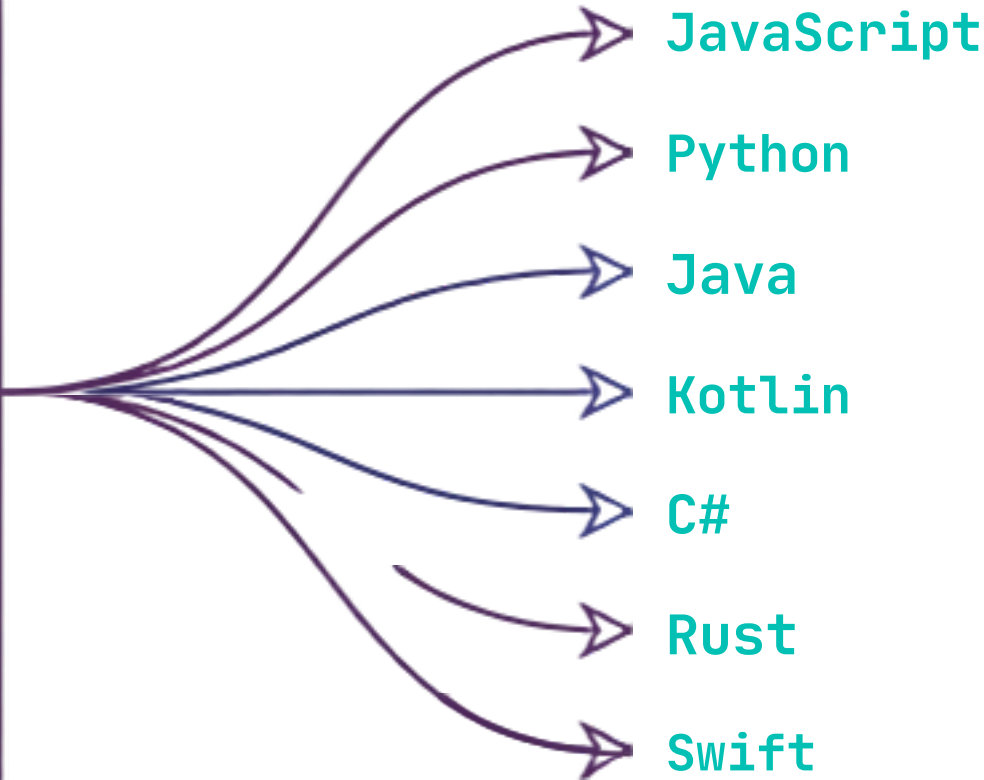
- Contexto: Proliferación de lenguajes funcionales sin un estándar común.
- 1987 (Portland): Formación de un comité internacional para crear un lenguaje funcional funcional abierto y estandarizado.
- Comité fundador: Simon Peyton Jones, Paul Hudak, Philip Wadler, John Hughes.
- Origen del nombre: En homenaje al matemático y lógico Haskell Brooks Curry.

Evolución e Influencia de Haskell



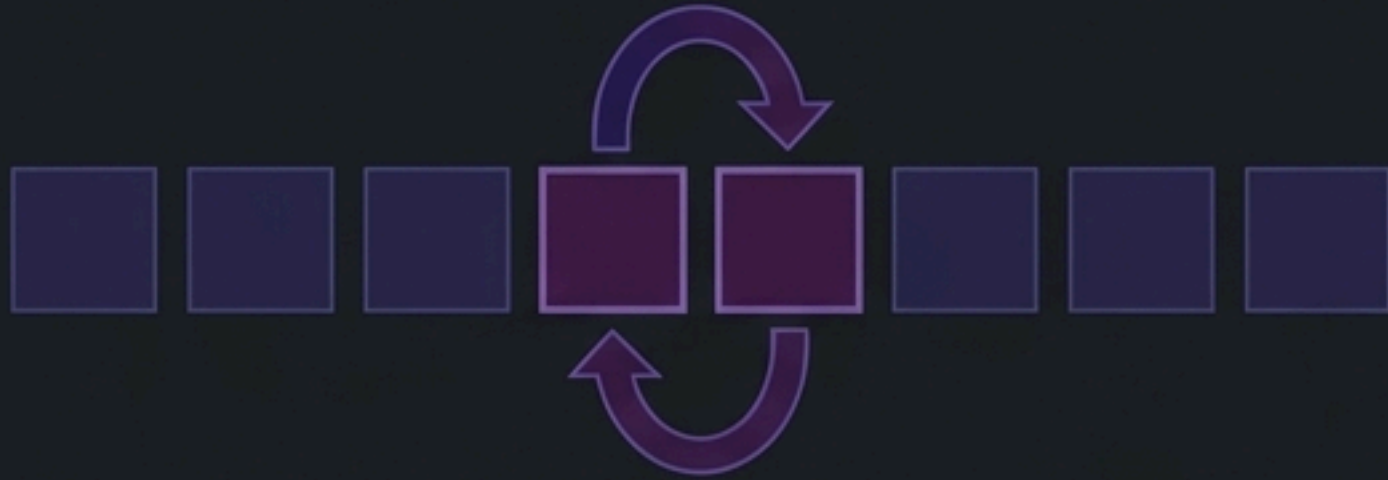
Características de Haskell:

Programación puramente funcional	Evaluación perezosa
Datos inmutables	Sistema de tipos estático y fuerte
Inferencia de tipos	Pattern matching
Tipos algebraicos	Transparencia referencial.



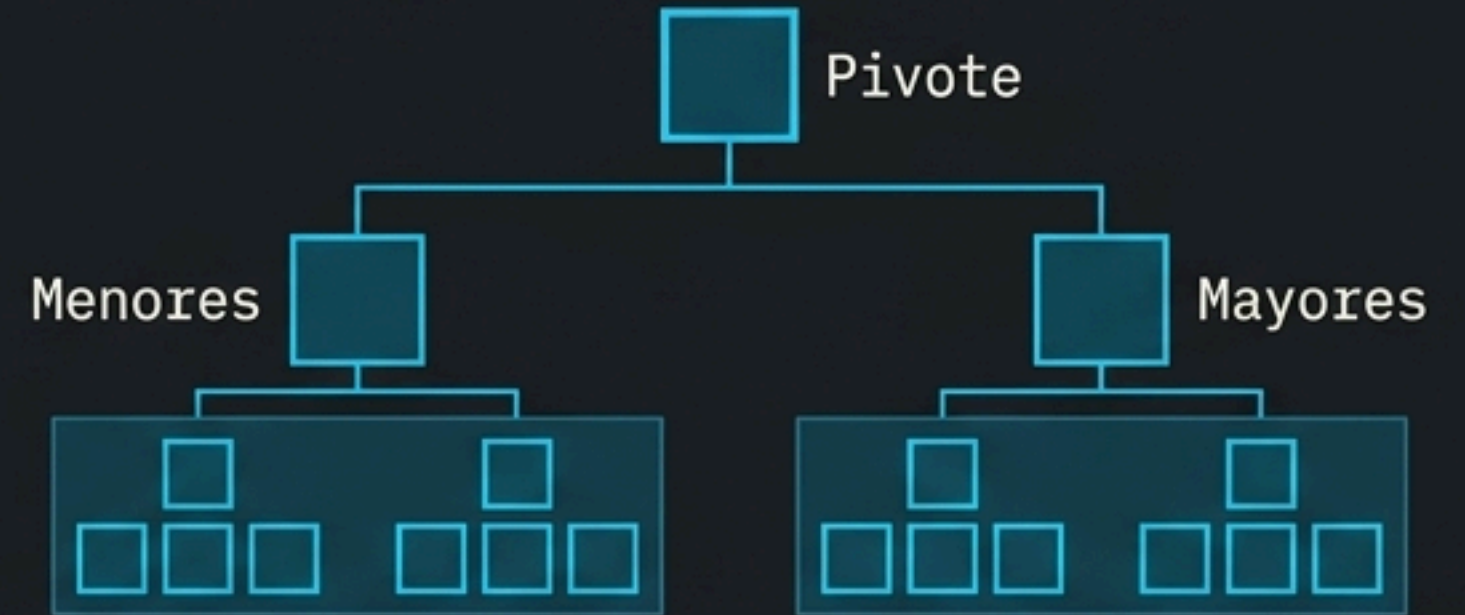
Algoritmos Elegidos

Bubble Sort



- Compara sistemáticamente elementos vecinos.
- Intercambia de posición los elementos desordenados.
- Repite pasadas por toda la lista hasta que no haya intercambios.
- Complejidad algorítmica habitual: $O(n^2)$

Quick Sort



- Elige un elemento central como pivote.
- Separa los elementos restantes en dos subgrupos: menores y mayores al pivote.
- Ordena recursivamente cada nueva partición.
- Generalmente es mucho más eficiente que Bubble Sort para listas grandes.

Implementación en Haskell y F#

Los algoritmos tienen exactamente el mismo objetivo, pero su implementación difiere drásticamente según la arquitectura fundamental de cada lenguaje.

Haskell (Paradigma Puro)

- **Estructura:** Listas enlazadas e inmutables.
- **Mecanismo:** Alta dependencia de la recursividad.
- **Memoria:** Creación constante de nuevas listas intermedias.
- **Ejecución:** Evaluación perezosa (Lazy Evaluation).
- **Quick Sort:** Basado en operaciones de filtrado y concatenación de listas.



F# (Paradigma Híbrido/.NET)

- **Estructura:** Arrays mutables nativos.
- **Mecanismo:** Modificación directa de la memoria.
- **Memoria:** Operaciones in-place (intercambios mediante una función swap).
- **Ejecución:** Evaluación estricta (inmediata).
- **Quick Sort:** Partición directa sobre el array original sin crear nuevas estructuras.

Metodología



Generación de datos

- Creación de listas de números aleatorios y desordenados.
- **Bubble Sort:** Volumen de prueba de 10.000 elementos.
- **Quick Sort:** Volumen de prueba de 100.000 elementos.



Ejecución del algoritmo

- **Haskell:** Compilado nativamente mediante GHC con el flag de máxima optimización -O2.
- **F#:** Ejecutado mediante el entorno .NET en configuración Release (sin herramientas de depuración).



Medición del tiempo

Registro del tiempo total de CPU utilizado exclusivamente para completar el ordenamiento.



Comparación

Análisis relativo de los deltas de rendimiento entre ambos compiladores.

Resultados Obtenidos

Bubble Sort (10.000 elementos)

Generating 10000 random elements...
Sorting...

Haskell

1,420433 segundos

F#

0,113954 segundos

F# fue aproximadamente
12,5 veces más rápido.

Sorted successfully! Time elapsed: 0.113954 seconds

Quick Sort (100.000 elementos)

Haskell

0,115578 segundos

F#

0,015147 segundos

F# fue aproximadamente
7,6 veces más rápido.

Sorted successfully! Time elapsed: 0.015147 seconds

Análisis de los Resultados

Modelo de Evaluación

La **evaluación perezosa** en Haskell frente a la **evaluación estricta** en F#. En Haskell, el procesamiento masivo genera una acumulación de expresiones pendientes (*thunks*) en la RAM.

Gestión de Memoria y Entorno

Uso de **listas inmutables** (Haskell) vs. **arrays mutables** (F#). La inmutabilidad fuerza la creación de nuevas listas; la mutabilidad permite operaciones **in-place**.
Adicionalmente, influyen las **optimizaciones de bajo nivel** de GHC frente al manejo de memoria de .NET.

Mecánica Algorítmica

El **Quick Sort** de Haskell requiere procesos costosos de **filtrado y concatenación**. F# resuelve la partición mediante **intercambios directos** en la **misma dirección de memoria**.

Diferencia de Rendimiento Favorable a F#

Nota analítica: Estas son posibles explicaciones arquitectónicas basadas en este caso de uso. No puede afirmarse de manera categórica que Haskell sea siempre un lenguaje más lento.

Conclusiones

01

Ventaja Absoluta en las Pruebas

F# obtuvo tiempos de ejecución consistentemente menores en ambas pruebas algorítmicas.

02

Impacto Relativo

La disparidad de rendimiento más dramática se observó en Bubble Sort, con una diferencia de 12,5x a favor de F#.

03

El Poder de la Mutabilidad

Los arrays mutables y las operaciones directas de intercambio (in-place) fueron la principal ventaja táctica de F#.

04

El Costo de la Pureza

La filosofía inmutable de Haskell generó una sobrecarga computacional debido a la creación constante de estructuras y listas intermedias.

05

Dependencia del Contexto

Los resultados obtenidos no permiten establecer como regla universal que F# sea siempre más rápido que Haskell.

06

Veredicto Final

El rendimiento depende fundamentalmente del algoritmo y de cómo interactúan la sintaxis y la arquitectura del lenguaje con el hardware subyacente.

Muchas gracias
¿Preguntas?